



K.N. Toosi University of Technology

OFFLINE JAVA GUIDE

Chapter 3

Regular Expressions

From zero to practical — learn what regular expressions are, how to read and write them, and how to use them effectively in Java with Pattern, Matcher, and real-world validation examples.

COURSE

Advanced Programming

SEMESTER

Spring 2026

DEPARTMENT

Computer Engineering

v1.0

Table of Contents

3.1	What is a Regular Expression?	Concept, purpose, basics
3.2	Regex Building Blocks	Character classes, quantifiers, anchors
3.3	Groups, Alternation & Escaping	Capturing, OR logic, special chars
3.4	Regex Cheatsheet	Complete quick-reference tables
3.5	Regex in Java: Pattern & Matcher	API overview, compilation
3.6	Java Regex Methods in Detail	matches, find, split, replace
3.7	Capturing Groups in Java	Extracting matched parts
3.8	Practical Examples	Email, phone, password patterns

3.1

What is a Regular Expression?

A **regular expression** (regex for short) is a sequence of characters that defines a **search pattern**. Think of it as a specialized mini-language for describing text patterns.

Why Learn Regex?

Regex lets you answer questions like:

- › Does this string look like a valid email address?
- › Extract all phone numbers from this document.
- › Replace all occurrences of "color" and "colour" with "color".
- › Is this password strong enough?
- › Split a string by one or more spaces, commas, or tabs.

Without regex, each of these requires manual loops and character-by-character checks. With regex, they become **one line**.

The Core Idea

A regex pattern is matched **against a text**. The engine scans the text and tells you:

QUESTION	JAVA METHOD
Does the entire text match the pattern?	<code>String.matches()</code>
Does any part of the text match?	<code>Matcher.find()</code>
Where are the matches? What did they capture?	<code>Matcher.start()</code> , <code>Matcher.group()</code>
Replace all matches	<code>String.replaceAll()</code>
Split text at match boundaries	<code>String.split()</code>

How to Read Regex

Regex uses special characters (metacharacters) to represent **categories** and **quantities** instead of literal characters. For example:

```
1 // Literal match - the pattern IS the exact text
2 "Hello"           matches "Hello"           but not "hello" or "Hello!"
3
4 // Digit pattern - \d means "any single digit"
5 "\\d\\d\\d"       matches "123"             but not "12" or "abc"
6
7 // Quantifier - \d{3} means "exactly 3 digits"
8 "\\d{3}"          matches "123" and "456"    but not "12" or "1234"
9
10 // Character class - [aeiou] means "any one vowel"
11 "[aeiou]"        matches "a", "e", "i"      but not "b" or "ab"
```

JAVA

In these examples, backslashes are **doubled** (`\\d` not `\d`) because Java string literals also use backslash as an escape character. Inside a Java string, `\d` is not valid — you must write `\\d`. This is a Java-specific rule, not a regex rule.

MENTAL MODEL

- › A regex is a **pattern description**, not code that runs
- › The regex engine is the program that reads your pattern and checks text against it
- › Most characters match themselves literally: `cat` matches `cat`
- › A handful of special characters (`.` `*` `+` `?` `[` `]` `\` `^` `$` `|` `(` `)` `{` `}`) have special meaning
- › Use `\` to escape a special character when you want its literal meaning

1. Literal Characters

Most characters in a regex match themselves exactly:

LITERALS

<code>cat</code>	matches <code>"cat"</code> – the letters c, a, t in sequence
<code>Hello123</code>	matches <code>"Hello123"</code> – letters and digits are literal

2. Character Classes — Match One Character from a Set

CHARACTER CLASSES

<code>[abc]</code>	any one of: a , b , or c
<code>[a-z]</code>	any lowercase letter from a to z
<code>[A-Z]</code>	any uppercase letter from A to Z
<code>[a-zA-Z]</code>	any letter (lowercase or uppercase)
<code>[0-9]</code>	any digit from 0 to 9
<code>[a-zA-Z0-9]</code>	any letter or digit (alphanumeric)
<code>[^abc]</code>	any character except a, b, c (^ = negation inside [])
<code>[^0-9]</code>	any character that is not a digit

3. Shorthand Character Classes

These are predefined classes for common categories. Note: in Java strings, always double the backslash.

SHORTHAND CLASSES (JAVA STRING FORM)

<code>\\d</code>	any digit – same as <code>[0-9]</code>
<code>\\D</code>	any non-digit – same as <code>[^0-9]</code>
<code>\\w</code>	any word character – same as <code>[a-zA-Z0-9_]</code>
<code>\\W</code>	any non-word character
<code>\\s</code>	any whitespace (space, tab, newline, etc.)
<code>\\S</code>	any non-whitespace character
<code>.</code>	any character except newline (the wildcard)

TIP

Mnemonic: lowercase = the thing itself, UPPERCASE = the opposite. `\\d` = digit, `\\D` = NOT digit. `\\s` = space, `\\S` = NOT space. `\\w` = word, `\\W` = NOT word.

4. Quantifiers — How Many Times?

Quantifiers apply to the **element immediately before them** (a character, a class, or a group).

QUANTIFIERS

<code>*</code>	zero or more – <code>ab*c</code> matches "ac", "abc", "abbc"
<code>+</code>	one or more – <code>ab+c</code> matches "abc", "abbc" but NOT "ac"
<code>?</code>	zero or one (optional) – <code>colou?r</code> matches "color" and "colour"
<code>{n}</code>	exactly n – <code>\d{4}</code> matches "2024" but not "24"
<code>{n,}</code>	n or more – <code>\d{2,}</code> matches "12", "123", "12345"
<code>{n,m}</code>	between n and m – <code>\d{2,4}</code> matches "12", "123", "1234"

5. Anchors — Position, Not Content

Anchors don't match characters — they match **positions** in the text.

ANCHORS

<code>^</code>	Start of string – <code>^Hello</code> matches "Hello" only at the beginning
<code>\$</code>	End of string – <code>world\$</code> matches "world" only at the end
<code>\b</code>	Word boundary – <code>\bcat\b</code> matches "cat" but not "catalog" or "scat"
<code>\B</code>	Non-word boundary – <code>\Bcat\B</code> matches "cat" inside "scatter"

Putting It Together

```
1 // "A word that is exactly 3 letters long"
2 "\\b[a-zA-Z]{3}\\b"
3 //  \b      = word boundary (start)
4 //  [a-zA-Z] = any letter
5 //  {3}     = exactly 3 times
6 //  \b      = word boundary (end)
7 // Matches: "cat", "Dog", "run" – but not "cats"
```

JAVA

```
1 // "A simple integer (optional sign + digits)"
2 "[+-]?\\d+"
3 //  [+]?   = optional + or -
4 //  \\d+   = one or more digits
5 // Matches: "42", "-7", "+100" – but not "3.14" or "abc"
```

JAVA

3.3 Groups, Alternation & Escaping

Capturing Groups — ()

Parentheses create a **group**. Groups serve two purposes: applying quantifiers to multiple characters, and **capturing** matched text for extraction.

```
GROUPS

(abc)+           matches "abc", "abcabc", "abcabcabc" — the + applies to the whole group
(\\d{2})-(\\d{2}) matches "12-34" and captures "12" as group 1, "34" as group 2
```

Non-Capturing Groups — (?:)

When you need grouping for quantifiers but **don't need to extract** the matched part:

```
NON-CAPTURING

(?:ab)+         matches "ab", "abab" — grouped but NOT captured
```

TIP Use non-capturing groups `(?: ... )` when you don't need to extract the group's content. It's slightly faster and keeps group numbering cleaner.

Alternation — | (OR)

The pipe `|` means **OR**. It matches either the left or right pattern.

```
ALTERNATION

cat|dog         matches "cat" OR "dog"
color|colour    matches either spelling
(red|green|blue) matches any of the three colors (grouped)
```

WARNING Alternation has **low precedence**. `cat|dog food` means `cat` OR `dog food`, not `cat food` OR `dog food`. Use groups: `(cat|dog) food`.

Escaping Special Characters

The metacharacters `. * + ? [ ] ( ) { } \ ^ $ |` have special meaning in regex. To match them **literally**, prefix with `\` (which becomes `\\` in Java strings):

ESCAPING (JAVA STRING FORM)

<code>\\.</code>	matches a literal dot (not "any character")
<code>\\\\</code>	matches a literal backslash (Java: <code>"\\\\"</code> → regex <code>"\\"</code> → text <code>\</code>)
<code>*</code>	matches a literal asterisk
<code>\\(</code>	matches a literal opening parenthesis
<code>\\\$</code>	matches a literal dollar sign

Greedy vs. Lazy Quantifiers

By default, quantifiers are **greedy** — they match as much as possible. Adding `?` after a quantifier makes it **lazy** — it matches as little as possible.

GREEDY VS LAZY

<code>.*</code>	Greedy: matches as much as possible
<code>.*?</code>	Lazy: matches as little as possible

```
1 String text = "<b>Hello</b> <b>World</b>";
2
3 // Greedy: matches from FIRST <b> to LAST </b>
4 "<b>.*</b>"
5 // → "<b>Hello</b> <b>World</b>" (swallowed everything in between)
6
7 // Lazy: matches from FIRST <b> to FIRST </b>
8 "<b>.*?</b>"
9 // → "<b>Hello</b>" (stops at first closing tag)
```

JAVA

QUICK REFERENCE — THE DOUBLE BACKSLASH RULE

- › Regex pattern: `\d{3}` — one backslash
- › Java string literal: `"\\d{3}"` — two backslashes (Java escaping)
- › Regex pattern: `\.` — escaped dot
- › Java string literal: `"\\."` — two backslashes
- › Regex pattern: `\\` — literal backslash
- › Java string literal: `"\\\\"` — four backslashes

A compact reference you can come back to. All patterns are shown in their **Java string form** (with double backslashes).

Complete Pattern Reference

PATTERN	MATCHES	EXAMPLE MATCH
<code>[abc]</code>	Any of a, b, c	<code>a</code> , <code>b</code>
<code>[^abc]</code>	Not a, b, or c	<code>d</code> , <code>1</code>
<code>[a-z]</code>	Lowercase letter range	<code>m</code> , <code>z</code>
<code>[a-zA-Z0-9]</code>	Any alphanumeric	<code>A</code> , <code>5</code> , <code>k</code>
<code>\d</code>	Any digit [0-9]	<code>7</code>
<code>\D</code>	Non-digit	<code>a</code> , <code>!</code>
<code>\w</code>	Word char [a-zA-Z0-9_]	<code>_</code> , <code>H</code>
<code>\W</code>	Non-word char	<code>@</code> , <code></code>
<code>\s</code>	Whitespace	<code></code> , <code>\t</code> , <code>\n</code>
<code>\S</code>	Non-whitespace	<code>a</code> , <code>5</code>
<code>.</code>	Any char except \n	<code>a</code> , <code>!</code> , <code>5</code>
<code>\.</code>	Literal dot	<code>.</code>

Quantifiers

PATTERN	MEANING	ON <code>A</code>	ON <code>ABC</code>
<code>a*</code>	Zero or more	<code>"", "a", "aaa"</code>	<code>"", "a"</code>
<code>a+</code>	One or more	<code>"a", "aaa"</code>	<code>"a"</code>
<code>a?</code>	Zero or one	<code>"", "a"</code>	<code>"", "a"</code>
<code>a{3}</code>	Exactly 3	<code>"aaa"</code>	—
<code>a{2,4}</code>	2 to 4	<code>"aa", "aaa", "aaaa"</code>	—
<code>a{2,}</code>	2 or more	<code>"aa", "aaa", ...</code>	—

Anchors & Boundaries

PATTERN	MATCHES
<code>^</code>	Start of string (or line in MULTILINE mode)
<code>\$</code>	End of string (or line in MULTILINE mode)
<code>\b</code>	Word boundary (between <code>\w</code> and <code>\W</code>)
<code>\B</code>	Non-word boundary

Groups & Logic

PATTERN	MATCHES
<code>(abc)</code>	Group "abc" (captures)
<code>(?:abc)</code>	Group "abc" (no capture)
<code>a b</code>	<code>a</code> OR <code>b</code>
<code>(a b)c</code>	<code>ac</code> OR <code>bc</code>
<code>\1</code>	Back-reference to group 1 (same text as group 1 matched)

Common Practical Patterns

GOAL	JAVA PATTERN
One or more digits	<code>"\d+"</code>
Optional sign + digits	<code>"[+-]?\d+"</code>
Integer or decimal	<code>"\d+(\.\d+)?"</code>
Simple word (letters only)	<code>"[a-zA-Z]+"</code>
Word boundary match	<code>"\b[a-zA-Z]+\b"</code>
One or more spaces	<code>"\s+"</code>
Comma-separated values	<code>"[^,]+"</code>
Nothing (empty string)	<code>"</code>
Anything at all	<code>".*"</code>

3.5

Regex in Java: Pattern & Matcher

Java provides regex support through two main classes in `java.util.regex`:

CLASS	ROLE
<code>Pattern</code>	The compiled regex . Immutable once created. Created via <code>Pattern.compile()</code> .
<code>Matcher</code>	The engine that performs matching against a specific input text. Created from a Pattern via <code>pattern.matcher(input)</code> .

The Standard Workflow

```
1 import java.util.regex.Pattern;
2 import java.util.regex.Matcher;
3
4 // Step 1: Compile the regex pattern
5 Pattern pattern = Pattern.compile("\\d+");
6
7 // Step 2: Create a matcher for your input text
8 Matcher matcher = pattern.matcher("I have 2 cats and 15 dogs");
9
10 // Step 3: Use the matcher to find, extract, or replace
11 while (matcher.find()) {
12     System.out.println("Found: " + matcher.group()
13                       + " at index " + matcher.start());
14 }
```

Found: 2 at index 7 Found: 15 at index 18

Why Compile the Pattern?

TIP

`Pattern.compile()` parses and compiles the regex once. If you use the same pattern multiple times (especially in a loop), compiling once and reusing the Pattern is **much faster** than calling `String.matches()` repeatedly, which recompiles every time.

Quick Alternative: String Methods

For simple cases, `String` has built-in regex methods that compile the pattern internally:

METHOD	EQUIVALENT PATTERN/MATCHER	NOTE
<code>s.matches(regex)</code>	<code>Pattern.compile(regex).matcher(s).matches()</code>	Must match the entire string
<code>s.split(regex)</code>	<code>Pattern.compile(regex).split(s)</code>	Returns <code>String[]</code>
<code>s.replaceAll(regex, repl)</code>	<code>Pattern.compile(regex).matcher(s).replaceAll(repl)</code>	Returns new String
<code>s.replaceFirst(regex, repl)</code>	<code>Pattern.compile(regex).matcher(s).replaceFirst(repl)</code>	Returns new String

`String.matches()` requires the **entire string** to match. It implicitly adds `^` and `$` around your pattern. If you want to find a match **anywhere inside** the string, use `Matcher.find()` instead.

Pattern Flags

Flags change how the regex engine interprets the pattern. Passed as a second argument to `compile()`:

FLAG	EFFECT	EXAMPLE
<code>Pattern.CASE_INSENSITIVE</code>	Ignore case	<code>"hello".matches("(?i)HELLO")</code> → true
<code>Pattern.MULTILINE</code>	<code>^</code> and <code>\$</code> match line start/end, not just string	Useful for multi-line input
<code>Pattern.DOTALL</code>	<code>.</code> matches everything including <code>\n</code>	Useful for matching across lines
<code>Pattern.COMMENTS</code>	Allows whitespace and <code>#</code> comments in pattern	Makes complex patterns readable

```
1 // Using a flag
2 Pattern p = Pattern.compile("hello", Pattern.CASE_INSENSITIVE);
3 System.out.println(p.matcher("HELLO").matches()); // true
4 System.out.println(p.matcher("HeLlO").matches()); // true
5
6 // Inline flag (embedded in the pattern)
7 System.out.println("HELLO".matches("(?i)hello")); // true
```

JAVA

Matcher.matches() — Entire String Match

Returns `true` only if the **entire** input matches the pattern:

```
1 Pattern p = Pattern.compile("\\d+");
2 Matcher m = p.matcher("12345");
3 System.out.println(m.matches()); // true
4
5 Matcher m2 = p.matcher("123abc");
6 System.out.println(m2.matches()); // false - "abc" is not a digit
7
8 // Equivalent via String:
9 System.out.println("12345".matches("\\d+")); // true
10 System.out.println("123abc".matches("\\d+")); // false
```

JAVA

Matcher.find() — Find Any Match

Scans the input for the **next** match (anywhere in the string). Returns `true` if found:

```
1 Pattern p = Pattern.compile("\\d+");
2 Matcher m = p.matcher("I have 2 cats and 15 dogs");
3
4 while (m.find()) {
5     System.out.println("Match: " + m.group()
6         + " | Start: " + m.start()
7         + " | End: " + m.end());
8 }
```

JAVA

Match: 2 | Start: 7 | End: 8 Match: 15 | Start: 18 | End: 20

OUTPUT

Matcher.looksAt() — Match from the Start

Like `matches()` but doesn't require the **entire** string to match — just the beginning:

```
1 Pattern p = Pattern.compile("\\d+");
2 System.out.println(p.matcher("123abc").looksAt()); // true - starts with digits
3 System.out.println(p.matcher("abc123").looksAt()); // false - doesn't start with digits
4 System.out.println(p.matcher("123abc").matches()); // false - entire string isn't digits
```

JAVA

METHOD	MUST MATCH ENTIRE STRING?	MUST START FROM BEGINNING?
<code>matches()</code>	Yes	Yes
<code>looksAt()</code>	No	Yes
<code>find()</code>	No	No

String.split() with Regex

JAVA

```
1 // Split by one or more spaces
2 String[] words = "Hello World Java".split("\\s+");
3 // ["Hello", "World", "Java"]
4
5 // Split by comma or semicolon
6 String[] parts = "red,green;blue,yellow".split("[,;]");
7 // ["red", "green", "blue", "yellow"]
8
9 // Split by digits
10 String[] tokens = "abc123def456ghi".split("\\d+");
11 // ["abc", "def", "ghi"]
12
13 // Limit the number of splits
14 String[] limited = "a,b,c,d,e".split(",", 3);
15 // ["a", "b", "c,d,e"]
```

String.replaceAll() and replaceFirst()

JAVA

```
1 String text = "Hello 123, world 456, test 789";
2
3 // Replace all digit sequences with "NUM"
4 String result1 = text.replaceAll("\\d+", "NUM");
5 System.out.println(result1);
6 // "Hello NUM, world NUM, test NUM"
7
8 // Replace only the first match
9 String result2 = text.replaceFirst("\\d+", "NUM");
10 System.out.println(result2);
11 // "Hello NUM, world 456, test 789"
```

Matcher.appendReplacement() and appendTail() — Advanced Replace

For replacements that need custom logic per match:

JAVA

```
1 String text = "I have 2 cats and 15 dogs";
2 Pattern p = Pattern.compile("\\d+");
3 Matcher m = p.matcher(text);
4 StringBuffer sb = new StringBuffer();
5
6 while (m.find()) {
7     int num = Integer.parseInt(m.group());
8     m.appendReplacement(sb, "[" + num + "]");
9 }
10 m.appendTail(sb);
11
12 System.out.println(sb);
13 // "I have [2] cats and [15] dogs"
```

Pattern.matches() — Static One-Liner

Groups don't just organize your pattern — they let you **extract** specific parts of each match.

Group Numbering

Groups are numbered starting from 1, left to left by their opening parenthesis. Group 0 is always the **entire match**.

```
1 Pattern p = Pattern.compile("(\\d{4})-(\\d{2})-(\\d{2})");
2 Matcher m = p.matcher("Date: 2024-01-15");
3
4 if (m.find()) {
5     System.out.println("Full match: " + m.group(0)); // "2024-01-15"
6     System.out.println("Year:      " + m.group(1)); // "2024"
7     System.out.println("Month:    " + m.group(2)); // "01"
8     System.out.println("Day:      " + m.group(3)); // "15"
9     System.out.println("Group count: " + m.groupCount()); // 3
10 }
```

JAVA

Full match: 2024-01-15 Year: 2024 Month: 01 Day: 15 Group count: 3

OUTPUT

Nested Groups

```
1 Pattern p = Pattern.compile("((\\w+)@(\\w+\\.\\w+))");
2 Matcher m = p.matcher("Contact: ali@gmail.com");
3
4 if (m.find()) {
5     System.out.println("Full email: " + m.group(0)); // "ali@gmail.com"
6     System.out.println("Group 1:    " + m.group(1)); // "ali@gmail.com"
7     System.out.println("Username:  " + m.group(2)); // "ali"
8     System.out.println("Domain:    " + m.group(3)); // "gmail.com"
9 }
```

JAVA

Full email: ali@gmail.com Group 1: ali@gmail.com Username: ali Domain: gmail.com

OUTPUT

Named Groups (Java 7+)

You can name groups with `(?<name>...)` and access them by name:


```

1 Pattern p = Pattern.compile("(?<year>\\d{4})-(?<month>\\d{2})-(?<day>\\d{2})");
2 Matcher m = p.matcher("2024-01-15");
3
4 if (m.find()) {
5     System.out.println(m.group("year"));    // "2024"
6     System.out.println(m.group("month"));   // "01"
7     System.out.println(m.group("day"));     // "15"
8 }

```

Extracting All Matches as a List

A common pattern — collect all matches into a list:

```

1 import java.util.ArrayList;
2 import java.util.List;
3
4 String text = "Prices: 120$, 45$, 300$, 8$";
5 Pattern p = Pattern.compile("\\d+");
6 Matcher m = p.matcher(text);
7
8 List<String> prices = new ArrayList<>();
9 while (m.find()) {
10     prices.add(m.group());
11 }
12 System.out.println(prices);    // [120, 45, 300, 8]

```

MISTAKE

Always call `m.find()` before calling `m.group()`. If no match has been found, `group()` throws `IllegalStateException`.
The `if (m.find())` or `while (m.find())` guard is essential.

Three real-world validation patterns, from simple to progressively more complex. Each includes the requirements, the pattern, and a line-by-line explanation.

Example 1 — Simple Email Validation

Requirements:

- One or more characters before the @
- Exactly one @ symbol
- One or more characters for the domain name
- A dot followed by 2 or more characters for the extension

JAVA

```
1  import java.util.regex.Pattern;
2  import java.util.regex.Matcher;
3
4  String emailPattern = "[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}";
5  Pattern p = Pattern.compile(emailPattern);
6
7  String[] tests = {
8      "ali@gmail.com",
9      "sara.student@kntu.ac.ir",
10     "reza_2024@yahoo.co.uk",
11     "invalid-email",
12     "@missing-local.com",
13     "no-domain@",
14     "spaces in@email.com"
15 };
16
17 for (String email : tests) {
18     Matcher m = p.matcher(email);
19     System.out.printf("%-30s → %s\n", email, m.matches() ? "VALID" : "INVALID");
20 }
```

OUTPUT

ali@gmail.com → VALID sara.student@kntu.ac.ir → VALID reza_2024@yahoo.co.uk → VALID invalid-email → INVALID
@missing-local.com → INVALID no-domain@ → INVALID spaces in@email.com → INVALID

Pattern Breakdown

PART	MEANING
[a-zA-Z0-9._%+-]+	Username: one or more letters, digits, dots, underscores, percent, plus, hyphen
@	The literal @ symbol
[a-zA-Z0-9.-]+	Domain: one or more letters, digits, dots, hyphens
\\.	A literal dot (escaped)
[a-zA-Z]{2,}	Extension: 2 or more letters (com, ir, uk, etc.)

This is a **practical** email pattern, not an RFC-compliant one. Real email validation is extremely complex. This pattern covers 99% of real-world cases while remaining readable — which is the right tradeoff for most applications.

Example 2 — Phone Number Validation

Requirements:

- Accept formats: `09123456789`, `0912-345-6789`, `+98 912 345 6789`, `+989123456789`
- Must start with 09 or +98
- Total of 10 digits after the prefix
- Dashes and spaces are optional and allowed in certain positions

```
1  String phonePattern = "(?:\\+98|0)9\\d{2}[- ]?\\d{3}[- ]?\\d{4}";
2  Pattern p = Pattern.compile(phonePattern);
3
4  String[] tests = {
5      "09123456789",
6      "0912-345-6789",
7      "+98912 345 6789",
8      "+989123456789",
9      "0912 345 6789",
10     "01234567890",      // wrong prefix
11     "0912345678",      // too short (9 digits after 09)
12     "091234567890",    // too long
13     "9123456789"       // missing leading 0
14 };
15
16 for (String phone : tests) {
17     Matcher m = p.matcher(phone);
18     System.out.printf("%-20s → %s%n", phone, m.matches() ? "VALID" : "INVALID");
19 }
```

JAVA

```
09123456789 → VALID 0912-345-6789 → VALID +98 912 345 6789 → VALID +989123456789 → VALID 0912 345 6789 →
VALID 01234567890 → INVALID 0912345678 → INVALID 091234567890 → INVALID 9123456789 → INVALID
```

OUTPUT

Pattern Breakdown

PART	MEANING
<code>(?:\\+98 0)</code>	Non-capturing group: either <code>+98</code> or <code>0</code>
<code>9</code>	Literal digit 9 (all Iranian mobile numbers have 9 here)
<code>\\d{2}</code>	Exactly 2 digits (the operator code: 12, 13, 30, etc.)
<code>[-]?</code>	Optional dash or space
<code>\\d{3}</code>	Exactly 3 digits
<code>[-]?</code>	Optional dash or space
<code>\\d{4}</code>	Exactly 4 digits

Example 3 — Strong Password Validation

Requirements:

- At least 8 characters long
- Contains at least one uppercase letter
- Contains at least one lowercase letter
- Contains at least one digit
- Contains at least one special character from `!@#$%^&*`

```
1  String passwordPattern = "^(?=.*[A-Z])(?=.*[a-z])(?=.*\\d)(?=.*[!@#$%^&*]).{8,}$";
2  Pattern p = Pattern.compile(passwordPattern);
3
4  String[] tests = {
5      "Abc123!@",           // valid - meets all requirements
6      "weakpass",           // invalid - no uppercase, digit, special
7      "ALLCAPS123!",        // invalid - no lowercase
8      "alllower123!",       // invalid - no uppercase
9      "NoSpecial123",       // invalid - no special character
10     "Short1!",             // invalid - only 7 characters
11     "PerfectPass#2024"    // valid
12 };
13
14 for (String pass : tests) {
15     Matcher m = p.matcher(pass);
16     System.out.printf("%-20s → %s%n", pass, m.matches() ? "VALID" : "INVALID");
17 }
```

```
Abc123!@ → VALID weakpass → INVALID ALLCAPS123! → INVALID alllower123! → INVALID NoSpecial123 → INVALID
Short1! → INVALID PerfectPass#2024 → VALID
```

PART	MEANING
<code>^</code>	Start of string (we need full-match control)
<code>(?=.*[A-Z])</code>	Lookahead: ensures at least one uppercase letter exists somewhere
<code>(?=.*[a-z])</code>	Lookahead: ensures at least one lowercase letter exists somewhere
<code>(?=.*\\d)</code>	Lookahead: ensures at least one digit exists somewhere
<code>(?=.*[!@#\$%^&*])</code>	Lookahead: ensures at least one special character exists somewhere
<code>.{8,}</code>	Any 8 or more characters (the actual match)
<code>\$</code>	End of string

How Lookaheads Work

The `(?= ... )` syntax is a **zero-width lookahead**. It checks if the pattern inside exists ahead, but **doesn't consume any characters**. Think of it as a checkpoint — the engine looks ahead, verifies the condition, then comes back and continues matching from the same position.

Lookahead flow for "Abc123!@"

<code>(?=.*[A-Z])</code>	Look ahead → found "A" ✓
<code>(?=.*[a-z])</code>	Look ahead → found "b", "c" ✓
<code>(?=.*\\d)</code>	Look ahead → found "1", "2", "3" ✓
<code>(?=.*[!@#\$%^&*])</code>	Look ahead → found "!", "@" ✓
<code>.{8,}</code>	Now actually match → "Abc123!@" (8 chars) ✓

TIP

The lookahead pattern is reusable. To add or remove requirements, just add or remove a `(?=.*[ ... ])` group. For example, to also require at least 12 characters instead of 8, change `.{8,}` to `.{12,}`.

Example 4 — Extracting Information from Text

Goal: From a log-style text, extract all email addresses and all numbers with their units.

```
1  String text = "Contact ali@gmail.com or sara@yahoo.com for info. "
2      + "Server load: 85%, memory: 4.2GB, disk: 256GB, uptime: 72hrs.";
3
4  // Extract all email addresses
5  Pattern emailP = Pattern.compile("[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}");
6  Matcher emailM = emailP.matcher(text);
7
8  System.out.println("Emails found:");
9  while (emailM.find()) {
10     System.out.println("  " + emailM.group());
11 }
12
13 // Extract all numbers (integers and decimals)
14 Pattern numP = Pattern.compile("\\d+\\.?\\d*");
15 Matcher numM = numP.matcher(text);
16
17 System.out.println("Numbers found:");
18 while (numM.find()) {
19     System.out.println("  " + numM.group() + " at index " + numM.start());
20 }
```

JAVA

```
Emails found: ali@gmail.com sara@yahoo.com Numbers found: 85 at index 63 4.2 at index 76 256 at index 89 72
at index 104
```

OUTPUT

QUICK REFERENCE — BUILDING A VALIDATION PATTERN

- › Start by listing the rules in plain language (like the requirements above)
- › Translate each rule to a regex fragment
- › For "must contain X" rules where X can appear anywhere → use lookaheads: `(?=.*X)`
- › For "must be exactly this format" → describe the format left to right
- › Always test with: valid inputs, invalid inputs, edge cases (empty string, very long input)
- › Use `^` and `$` when validating the entire string; omit them when searching within text
- › Keep patterns readable — add comments using `Pattern.COMMENTS` flag for complex ones

This document was compiled from the following resources. All content was adapted and restructured for educational use.

[1] **Programiz — Java Regular Expressions**

Regex syntax, Pattern and Matcher usage, practical examples

<https://www.programiz.com/java-programming/regex>

[2] **GeeksforGeeks — Regular Expressions in Java**

Pattern classes, matcher methods, capturing groups, validation patterns

<https://www.geeksforgeeks.org/regular-expressions-in-java/>

[3] **Oracle — Java Documentation — `java.util.regex.Pattern`**

Official API reference for Pattern class, syntax summary, flag definitions

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/regex/Pattern.html>

[4] **Regular-Expressions.info**

Comprehensive regex tutorial covering syntax, flavors, and techniques

<https://www.regular-expressions.info/>

[5] **Baeldung — Java Regex Guide**

Practical guides on Pattern/Matcher, email validation, password patterns

<https://www.baeldung.com/regular-expressions-java>

CREATED BY

Sepehr Ghardashi

sepehr.ghardashi@gmail.com